Подготовка выполняется владельцем отдельной учебной базы MyDB. Пользователи Reader и Writer создаются без входа на сервер и проверяются через EXECUTE AS USER. Каждый блок запускается по порядку. Описанные ниже результаты ожидаемые; для подтверждения нужно выполнить запросы в SSMS.

```
CREATE DATABASE MyDB;
GO
USE MyDB;
GO
CREATE TABLE dbo.Groups (
 GroupID int PRIMARY KEY,
 GroupName nvarchar(50)
);
INSERT dbo.Groups VALUES (1,N'Группа А'),(2,N'Группа Б');
CREATE TABLE dbo.Students (
 StudentID int PRIMARY KEY,
 FullName nvarchar(100),
 Age int,
 GroupID int
);
INSERT dbo.Students VALUES
 (1,N'Иванов Иван',20,1),
 (2,N'Петров Пётр',21,1),
 (3,N'Сидоров Сидор',22,2),
 (4,N'Кузнецова Анна',19,2);
CREATE TABLE dbo.Orders (
 OrderID int PRIMARY KEY,
 CustomerID int,
 OrderDate datetime,
 Total decimal(10,2),
 Status nvarchar(20)
);
INSERT dbo.Orders VALUES
 (100,1,'20250801',500,N'Pending'),
 (101,2,'20250802',300,N'Pending'),
 (102,1,'20250803',700,N'Completed');
CREATE TABLE dbo.Employees (
 EmployeeID int PRIMARY KEY,
 FullName nvarchar(100),
```

```
 Department nvarchar(50),
 Salary decimal(10,2)
);
INSERT dbo.Employees VALUES
 (1,N'Иванов И.И.',N'Отдел продаж',50000),
 (2,N'Петров П.П.',N'Бухгалтерия',45000),
 (3,N'Сидоров С.С.',N'IT',60000);
GO
CREATE VIEW dbo.StudentsPublic AS
 SELECT FullName,GroupID FROM dbo.Students;
GO
CREATE VIEW dbo.EmployeesPublic AS
 SELECT EmployeeID,FullName,Department FROM dbo.Employees;
GO
CREATE VIEW dbo.MyOrders AS
 SELECT OrderID,CustomerID,OrderDate,Total,Status
 FROM dbo.Orders WHERE CustomerID=1
 WITH CHECK OPTION;
GO
```

1. SQL-код решения.

```
CREATE USER Reader WITHOUT LOGIN;
CREATE USER Writer WITHOUT LOGIN;
GRANT SELECT ON dbo.Students TO Reader;
GRANT INSERT ON dbo.Students TO Writer;
GO
EXECUTE AS USER='Reader';
SELECT * FROM dbo.Students;
BEGIN TRY
 INSERT dbo.Students VALUES (10,N'Test',20,1);
END TRY
BEGIN CATCH
 SELECT ERROR_NUMBER() AS ErrorNumber,
      ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
GO
EXECUTE AS USER='Writer';
INSERT dbo.Students VALUES (11,N'Writer Test',21,1);
BEGIN TRY
 SELECT * FROM dbo.Students;
END TRY
BEGIN CATCH
 SELECT ERROR_NUMBER() AS ErrorNumber,
      ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
GO
```

Reader сможет прочитать четыре исходные строки, но вставка будет запрещена. Writer сможет добавить строку с StudentID=11, но чтение будет запрещено. Права SELECT и INSERT выданы разным пользователям.

2. SQL-код решения.

```
REVOKE SELECT ON dbo.Students FROM Reader;
GRANT SELECT ON dbo.StudentsPublic TO Reader;
GO
EXECUTE AS USER='Reader';
SELECT * FROM dbo.StudentsPublic;
BEGIN TRY
 SELECT * FROM dbo.Students;
END TRY
BEGIN CATCH
 SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
GO
```

Представление покажет только FullName и GroupID, включая добавленного студента. Прямое чтение Students будет запрещено. Предыдущее разрешение обязательно отзываем. Представление и таблица принадлежат одному владельцу, поэтому отдельное разрешение на чтение таблицы через представление не требуется.

3. SQL-код решения.

```
GRANT INSERT ON dbo.Orders TO Writer;
DENY DELETE ON dbo.Orders TO Writer;
GO
EXECUTE AS USER='Writer';
INSERT dbo.Orders VALUES
 (103,1,GETDATE(),500,N'Pending');
BEGIN TRY
 DELETE dbo.Orders WHERE OrderID=100;
END TRY
BEGIN CATCH
 SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
GO
SELECT * FROM dbo.Orders WHERE OrderID IN (100,103);
GO
```

Вставка заказа 103 должна выполниться, удаление — завершиться отказом в доступе. Проверка от владельца покажет исходный заказ 100 и новый 103. Используем 103, потому что OrderID=100 уже занят; право SELECT для Writer в этом задании не выдано.

4. SQL-код решения.

```
GRANT UPDATE (Status) ON dbo.Orders TO Writer;
GRANT SELECT (OrderID) ON dbo.Orders TO Writer;
GO
EXECUTE AS USER='Writer';
UPDATE dbo.Orders SET Status=N'Completed'
 WHERE OrderID=100;
BEGIN TRY
 UPDATE dbo.Orders SET Total=999
  WHERE OrderID=100;
END TRY
BEGIN CATCH
 SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
GO
SELECT OrderID,Total,Status FROM dbo.Orders
 WHERE OrderID=100;
GO
```

Status заказа 100 изменится на Completed, а Total останется 500. UPDATE выдан только на Status. Чтение OrderID разрешено для условия WHERE; полного доступа к таблице это не даёт.

5. SQL-код решения.

```
GRANT SELECT ON dbo.EmployeesPublic TO Reader;
GO
EXECUTE AS USER='Reader';
SELECT * FROM dbo.EmployeesPublic;
BEGIN TRY
 SELECT * FROM dbo.Employees;
END TRY
BEGIN CATCH
 SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
GO
```

Представление покажет три строки с EmployeeID, FullName и Department без Salary. Прямое чтение Employees будет запрещено, так как разрешение выдано только на представление.

6. SQL-код решения.

```
EXECUTE AS USER='Reader';
BEGIN TRY
 EXEC(N'CREATE PROCEDURE dbo.TestProc
 AS SELECT "Hello";');
END TRY
BEGIN CATCH
 SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
GO
SELECT OBJECT_ID(N'dbo.TestProc',N'P') AS ProcedureID;
GO
```

CREATE PROCEDURE должен завершиться отказом в доступе, а ProcedureID будет NULL. У Reader нет права создания процедур. Динамический запрос позволяет корректно выполнить CREATE PROCEDURE отдельным пакетом и обработать ошибку.

7. SQL-код решения.

```
REVOKE INSERT ON dbo.Orders FROM Writer;
REVOKE UPDATE (Status) ON dbo.Orders FROM Writer;
REVOKE SELECT (OrderID) ON dbo.Orders FROM Writer;
GRANT SELECT, UPDATE ON dbo.MyOrders TO Writer;
GO
EXECUTE AS USER='Writer';
SELECT * FROM dbo.MyOrders;
BEGIN TRY
 SELECT * FROM dbo.Orders;
END TRY
BEGIN CATCH
 SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
GO
```

Writer увидит заказы 100, 102 и 103 с CustomerID=1. Чтение Orders напрямую будет запрещено. Прямые разрешения из предыдущих задач отзываются, чтобы нельзя было обойти представление. WITH CHECK OPTION не позволяет изменить заказ через представление так, чтобы его CustomerID перестал быть равен 1. В этом учебном примере Writer соответствует одному клиенту; для разных клиентов нужны отдельные правила фильтрации.

8. SQL-код решения.

```
REVOKE UPDATE ON dbo.MyOrders FROM Writer;
GO
SELECT OrderID,Total FROM dbo.Orders
 WHERE OrderID=100;
BEGIN TRAN;
EXECUTE AS USER='Writer';
BEGIN TRY
 EXEC(N'UPDATE dbo.Orders SET Total=9999
 WHERE OrderID=100;');
END TRY
BEGIN CATCH
 SELECT ERROR_MESSAGE() AS ErrorMessage;
END CATCH;
REVERT;
IF @@TRANCOUNT>0 ROLLBACK TRAN;
SELECT OrderID,Total FROM dbo.Orders
 WHERE OrderID=100;
SELECT @@TRANCOUNT AS OpenTransactions;
GO
```

Изменение Total должно завершиться отказом в доступе. После явного ROLLBACK сумма заказа 100 останется 500, OpenTransactions будет 0. Отказ в доступе сам по себе не гарантирует откат всей открытой транзакции, поэтому откат выполняется явно.

Документация: https://learn.microsoft.com/en-us/sql/t-sql/statements/grant-object-permissions-transact-sql ; https://learn.microsoft.com/en-us/sql/relational-databases/security/permissions-database-engine .